

Questions

Set A:

Design the **Movie** class in such a way that the following tester code produces the expected output. You are not allowed to modify the tester code. Use your judgment to add any necessary instance variables, constructors, methods, and helper classes if needed.

A Movie can have up to 15 scenes.

Tester Code	Expected Output
<pre> public class MovieTester { public static void main(String[] args) { Movie m1 = new Movie("Inception"); Movie m2 = new Movie("Interstellar", "Sci-Fi"); m1.display(); m2.display(); System.out.println("====="); m1.addScene("Dream", 18, true); m1.addScene("Kick", 12); m2.addScene("Docking", 15, true); m2.addScene("No Time For Caution", 11, true); m2.addScene("Cornfield"); System.out.println("====="); m1.display(); m2.display(); System.out.println("====="); m2.setPrequel(m1); m1.rateScene("Dream", 9.3); m2.rateScene("Docking", 9.8); m2.rateScene("No Time For Caution", 9.6); m2.rateScene("Cornfield", 8.6); m1.display(); m2.display(); System.out.println("====="); m2.totalImportantDuration(); m2.bestScene(); } } </pre>	<pre> Movie: Inception Genre: Unknown Prequel: None Movie: Interstellar Genre: Sci-Fi Prequel: None ===== Added: Dream Duration: 18 min Important: Yes Added: Kick Duration: 12 min Important: No Added: Docking Duration: 15 min Important: Yes Added: No Time For Caution Duration: 11 min Important: Yes Added: Cornfield Duration: 20 min Important: No ===== Movie: Inception Genre: Unknown Prequel: None Dream Rating: 0.0/10 Kick Rating: 0.0/10 Movie: Interstellar Genre: Sci-Fi Prequel: None Docking Rating: 0.0/10 No Time For Caution Rating: 0.0/10 Cornfield Rating: 0.0/10 ===== Movie: Inception Genre: Unknown Prequel: None Dream Rating: 9.3/10 Kick Rating: 0.0/10 Movie: Interstellar Genre: Sci-Fi Prequel: Inception Docking Rating: 9.8/10 No Time For Caution Rating: 9.6/10 Cornfield Rating: 8.6/10 ===== Total Important Duration of Interstellar: 26 min Best Scene of Interstellar: Docking (9.8/10) </pre>

Set B:

Design the **Playlist** class in such a way that the following tester code produces the expected output. You are not allowed to modify the tester code. Use your judgment to add any necessary instance variables, constructors, methods, and helper classes if needed.

A Playlist can contain up to 20 songs.

Tester Code	Expected Output
<pre>public class PlaylistTester { public static void main(String[] args) { Playlist p1 = new Playlist("Road Trip"); Playlist p2 = new Playlist("Workout", "Pop"); p1.display(); p2.display(); System.out.println("====="); p1.addSong("Believer", 3, true); p1.addSong("Thunder", 4); p2.addSong("Stronger", 5, true); p2.addSong("Till I Collapse", 4, true); p2.addSong("Champion"); System.out.println("====="); p1.display(); p2.display(); System.out.println("====="); p2.setPrevious(p1); p1.rateSong("Believer", 9.1); p2.rateSong("Stronger", 9.6); p2.rateSong("Till I Collapse", 9.3); p2.rateSong("Champion", 8.4); p1.display(); p2.display(); System.out.println("====="); p2.favoriteDuration(); p2.topRatedSong(); } }</pre>	<pre>Playlist: Road Trip Genre: Unknown Previous: None Playlist: Workout Genre: Pop Previous: None ===== Added: Believer Duration: 3 min Favorite: Yes Added: Thunder Duration: 4 min Favorite: No Added: Stronger Duration: 5 min Favorite: Yes Added: Till I Collapse Duration: 4 min Favorite: Yes Added: Champion Duration: 4 min Favorite: No ===== Playlist: Road Trip Genre: Unknown Previous: None Believer Rating: 0.0/10 Thunder Rating: 0.0/10 Playlist: Workout Genre: Pop Previous: None Stronger Rating: 0.0/10 Till I Collapse Rating: 0.0/10 Champion Rating: 0.0/10 ===== Playlist: Road Trip Genre: Unknown Previous: None Believer Rating: 9.1/10 Thunder Rating: 0.0/10 Playlist: Workout Genre: Pop Previous: Road Trip Stronger Rating: 9.6/10 Till I Collapse Rating: 9.3/10 Champion Rating: 8.4/10 ===== Favorite Song Duration of Workout: 9 min Top Rated Song of Workout: Stronger (9.6/10)</pre>

Solutions

Set A:

```
public class Movie {
    String title;
    String genre = "Unknown";
    Movie prequel;
    String[] sceneNames = new String[15];
    int[] durations = new int[15];
    boolean[] important = new boolean[15];
    double[] ratings = new double[15];
    int count = 0;

    public Movie(String title) {
        this.title = title;
    }
    public Movie(String title, String genre) {
        this.title = title;
        this.genre = genre;
    }
    public void addScene(String scene) {
        addScene(scene, 20, false);
    }
    public void addScene(String scene, int duration) {
        addScene(scene, duration, false);
    }
    public void addScene(String scene, int duration, boolean isImportant) {
        sceneNames[count] = scene;
        durations[count] = duration;
        important[count] = isImportant;
        ratings[count] = 0.0;
        System.out.println("Added: " + scene + " | Duration: " + duration + " min | Important: " + (isImportant ? "Yes" : "No"));
        count++;
    }
    public void setPrequel(Movie m) {
        prequel = m;
    }
    public void rateScene(String scene, double rating) {
        for (int i = 0; i < count; i++) {
            if (sceneNames[i].equals(scene)) {
                ratings[i] = rating;
                break;
            }
        }
    }
    public void display() {
        System.out.print("Movie: " + title + " | Genre: " + genre + " | Prequel: ");
        if (prequel == null)
            System.out.println("None");
        else
            System.out.println(prequel.title);

        for (int i = 0; i < count; i++) {
            System.out.println(sceneNames[i] + " | Rating: " + ratings[i] + "/10");
        }
    }
}
```

```

    }
}
public void totalImportantDuration() {
    int total = 0;

    for (int i = 0; i < count; i++) {
        if (important[i]) {
            total += durations[i];
        }
    }
    System.out.println("Total Important Duration of " + title + ": " + total + " min");
}
public void bestScene() {
    int max = 0;

    for (int i = 1; i < count; i++) {
        if (ratings[i] > ratings[max]) {
            max = i;
        }
    }
    System.out.println("Best Scene of " + title + ": " + sceneNames[max] +
        " (" + ratings[max] + "/10)");
}
}

```

Set B:

```
public class Playlist {
    String name;
    String genre = "Unknown";
    Playlist previous;

    String[] songNames = new String[20];
    int[] durations = new int[20];
    boolean[] favorite = new boolean[20];
    double[] ratings = new double[20];
    int count = 0;
    public Playlist(String name) {
        this.name = name;
    }
    public Playlist(String name, String genre) {
        this.name = name;
        this.genre = genre;
    }
    public void addSong(String song) {
        addSong(song, 4, false);
    }
    public void addSong(String song, int duration) {
        addSong(song, duration, false);
    }
    public void addSong(String song, int duration, boolean isFavorite) {
        songNames[count] = song;
        durations[count] = duration;
        favorite[count] = isFavorite;
        ratings[count] = 0.0;

        System.out.println("Added: " + song + " | Duration: " + duration + " min |
Favorite: " + (isFavorite ? "Yes" : "No"));
        count++;
    }
    public void setPrevious(Playlist p) {
        previous = p;
    }
    public void rateSong(String song, double rating) {
        for (int i = 0; i < count; i++) {
            if (songNames[i].equals(song)) {
                ratings[i] = rating;
                break;
            }
        }
    }
    public void display() {
        System.out.print("Playlist: " + name + " | Genre: " + genre + " | Previous: ");
        if (previous == null)
            System.out.println("None");
        else
            System.out.println(previous.name);
        for (int i = 0; i < count; i++) {
            System.out.println(songNames[i] + " | Rating: " + ratings[i] + "/10");
        }
    }
}
```

```

    }
    public void favoriteDuration() {
        int total = 0;
        for (int i = 0; i < count; i++) {
            if (favorite[i]) {
                total += durations[i];
            }
        }
        System.out.println("Favorite Song Duration of " + name + ": " + total + " min");
    }
    public void topRatedSong() {
        int max = 0;
        for (int i = 1; i < count; i++) {
            if (ratings[i] > ratings[max]) {
                max = i;
            }
        }
        System.out.println("Top Rated Song of " + name +
            ": " + songNames[max] +
            " (" + ratings[max] + "/10)");
    }
}

```

Rubric

Criteria	Marks
Class implementation and correct instance variables	2 marks
Constructors	- correctly create both overloaded constructors (0.5 + 0.5)
addScene()/addSong()	- correctly create both overloaded methods with proper parameters (1) - correctly store the object with appropriate default values (1)
rateScene()/rateSong()	- correctly create method with proper parameters (0.5) - correctly update the rating of the specified object (0.5)
setPrequel()/setPrevious()	- correctly create method with proper parameter (0.5) - correctly update the reference object (0.5)
display()	- correctly create method (0.5) - correctly print all required information in the expected format (0.5)
totalImportantDuration()/favoriteDuration()	- correctly create method (0.5) - correctly calculate and print the total duration by traversing all matching objects (0.5)
bestScene()/topRatedSong()	- correctly create method (0.5) - correctly identify and print the highest-rated object (0.5)